

EA-0005 - AQELYN Knowledge Graph

EA-0005 - AQELYN Knowledge Graph

Implementation Specification: IS-005

Publication Standard: FULL_COMPLETE

Brand: AQELYN

Status: Regenerated and normalized for AQELYN Platform Architecture Baseline v1.0

001 - Document Control

This Engineering Archive defines the approved implementation baseline for **AQELYN Knowledge Graph**. It replaces earlier AQELYN draft material and is the authoritative source for implementation, testing, review, and traceability.

The archive belongs to the AQELYN Platform architecture baseline and shall be implemented in the sequence assigned by the master roadmap. The repository structure remains fixed and may not be redesigned by developers or AI coding tools.

| Field | Value |

|---|---|

| Engineering Archive | EA-0005 |

| Specification | IS-005 |

| Module | AQELYN Knowledge Graph |

| Source of Truth | Master Markdown |

| Derived Artifacts | PDF, HTML, README, manifest, matrices, diagrams |

Implementation note: AQELYN Knowledge Graph must treat every decision as auditable. If the engine produces a finding, state transition, recommendation, event, or policy result, that output must reference the reason, source, timestamp, and confidence context. This supports AQELYN's user promise that security outcomes are understandable to non-experts and actionable by experts.

Quality note: AQELYN Knowledge Graph shall include code-level tests, schema-level tests, and documentation-level traceability. Any deviation discovered during implementation shall be raised as an Engineering Change Request rather than silently changing the architecture.

Operational note: AQELYN Knowledge Graph shall expose health state, readiness state, dependency state, and degraded-mode state. Operators must be able to understand whether the engine is fully operational, partially degraded, blocked by an upstream dependency, or intentionally disabled by policy.

Developer note: implementation of AQELYN Knowledge Graph shall follow clean boundaries between domain logic, infrastructure adapters, API handlers, persistence adapters, and event adapters. This is required so Codex, Claude Code, and human developers can implement and test the module incrementally without hidden coupling.

User-output note: any user-visible output related to AQELYN Knowledge Graph shall be written in clear language first and detailed technical language second. The system must always answer: what was found, why it matters, how AQELYN knows, and what the user can do next.

Government and enterprise note: AQELYN Knowledge Graph shall support auditability, exportable records, retention policy alignment, and review-ready evidence packages. The design supports private users, companies, managed service providers, and government environments without creating separate architectures.

002 - Executive Summary

AQELYN Knowledge Graph provides relationship modeling, graph storage, entity correlation, dependency analysis, reasoning support, and explainable context. It is a foundational AQELYN capability and must be implemented with deterministic behavior, strict interface discipline, auditable evidence, and high operational reliability.

The module contributes to the platform goal: make cybersecurity understandable, evidence-based, and actionable for home users, companies, and governments. Every output produced by this module must support the product principles: Explain Before Recommend, Simplicity First, Evidence Before Opinion, Human-Centered Security, and Expert Depth on Demand.

Implementation note: AQELYN Knowledge Graph must treat every decision as auditable. If the engine produces a finding, state transition, recommendation, event, or policy result, that output must reference the reason, source, timestamp, and confidence context. This supports AQELYN's user promise that security outcomes are understandable to non-experts and actionable by experts.

Quality note: AQELYN Knowledge Graph shall include code-level tests, schema-level tests, and documentation-level traceability. Any deviation discovered during implementation shall be raised as an Engineering Change Request rather than silently changing the architecture.

Operational note: AQELYN Knowledge Graph shall expose health state, readiness state, dependency state, and degraded-mode state. Operators must be able to understand whether the engine is fully operational, partially degraded, blocked by an upstream dependency, or intentionally disabled by policy.

Developer note: implementation of AQELYN Knowledge Graph shall follow clean boundaries between domain logic, infrastructure adapters, API handlers, persistence adapters, and event adapters. This is required so Codex, Claude Code, and human developers can implement and test the module incrementally without hidden coupling.

User-output note: any user-visible output related to AQELYN Knowledge Graph shall be written in clear language first and detailed technical language second. The system must always answer: what was found, why it matters, how AQELYN knows, and what the user can do next.

Government and enterprise note: AQELYN Knowledge Graph shall support auditability, exportable records, retention policy alignment, and review-ready evidence packages. The design supports private users, companies, managed service providers, and government environments without creating separate architectures.

003 - Vision and Purpose

The vision of AQELYN Knowledge Graph is to provide a stable, understandable, and secure capability that other AQELYN engines can depend on. The module shall be designed for local-first operation where possible and enterprise-scale operation where required.

The purpose of this archive is to transform the original IS-005 concept into an implementation-ready engineering document that Codex, Claude Code, human developers, and reviewers can use directly.

Implementation note: AQELYN Knowledge Graph must treat every decision as auditable. If the engine produces a finding, state transition, recommendation, event, or policy result, that output must reference the reason, source, timestamp, and confidence context. This supports AQELYN's user promise that security outcomes are understandable to non-experts and actionable by experts.

Quality note: AQELYN Knowledge Graph shall include code-level tests, schema-level tests, and documentation-level traceability. Any deviation discovered during implementation shall be raised as an Engineering Change Request rather than silently changing the architecture.

Operational note: AQELYN Knowledge Graph shall expose health state, readiness state, dependency state, and degraded-mode state. Operators must be able to understand whether the engine is fully operational, partially degraded, blocked by an upstream dependency, or intentionally disabled by policy.

Developer note: implementation of AQELYN Knowledge Graph shall follow clean boundaries between domain logic, infrastructure adapters, API handlers, persistence adapters, and event adapters. This is required so Codex, Claude Code, and human developers can implement and test the module incrementally without hidden coupling.

User-output note: any user-visible output related to AQELYN Knowledge Graph shall be written in clear language first and detailed technical language second. The system must always answer: what was found, why it matters, how AQELYN knows, and what the user can do next.

Government and enterprise note: AQELYN Knowledge Graph shall support auditability, exportable records, retention policy alignment, and review-ready evidence packages. The design supports private users, companies, managed service providers, and government environments without creating separate architectures.

004 - Scope and Boundaries

In scope: implementation behavior, data contracts, lifecycle states, events, evidence expectations, API boundaries, testing requirements, and operational controls for Knowledge Graph.

Out of scope: redesigning AQELYN platform architecture, changing repository hierarchy, replacing upstream or downstream engines, and introducing unapproved data models outside the Universal Object Model.

Implementation note: AQELYN Knowledge Graph must treat every decision as auditable. If the engine produces a finding, state transition, recommendation, event, or policy result, that output must reference the reason, source, timestamp, and confidence context. This supports AQELYN's user promise that security outcomes are understandable to non-experts and actionable by experts.

Quality note: AQELYN Knowledge Graph shall include code-level tests, schema-level tests, and documentation-level traceability. Any deviation discovered during implementation shall be raised as an Engineering Change Request rather than silently changing the architecture.

Operational note: AQELYN Knowledge Graph shall expose health state, readiness state, dependency state, and degraded-mode state. Operators must be able to understand whether the engine is fully operational, partially degraded, blocked by an upstream dependency, or intentionally disabled by policy.

Developer note: implementation of AQELYN Knowledge Graph shall follow clean boundaries between domain logic, infrastructure adapters, API handlers, persistence adapters, and event adapters. This is required so Codex, Claude Code, and human developers can implement and test the module incrementally without hidden coupling.

User-output note: any user-visible output related to AQELYN Knowledge Graph shall be written in clear language first and detailed technical language second. The system must always answer: what was found, why it matters, how AQELYN knows, and what the user can do next.

Government and enterprise note: AQELYN Knowledge Graph shall support auditability, exportable records, retention policy alignment, and review-ready evidence packages. The design supports private users, companies, managed service providers, and government environments without creating separate architectures.

005 - Engineering Objectives

Engineering Objectives for AQELYN Knowledge Graph shall be implemented in accordance with AQELYN Platform Architecture Baseline v1.0. The implementation must preserve traceability, deterministic behavior, explainability, and evidence-backed outputs.

This section defines engineering expectations for knowledge graph and provides the implementation team with stable guidance that should not be bypassed by local optimization or AI-generated redesign.

Implementation note: AQELYN Knowledge Graph must treat every decision as auditable. If the engine produces a finding, state transition, recommendation, event, or policy result, that output must reference the reason, source, timestamp, and confidence context. This supports AQELYN's user promise that security outcomes are understandable to non-experts and actionable by experts.

Quality note: AQELYN Knowledge Graph shall include code-level tests, schema-level tests, and documentation-level traceability. Any deviation discovered during implementation shall be raised as an Engineering Change Request rather than silently changing the architecture.

Operational note: AQELYN Knowledge Graph shall expose health state, readiness state, dependency state, and degraded-mode state. Operators must be able to understand whether the engine is fully operational, partially degraded, blocked by an upstream dependency, or intentionally disabled by policy.

Developer note: implementation of AQELYN Knowledge Graph shall follow clean boundaries between domain logic, infrastructure adapters, API handlers, persistence adapters, and event adapters. This is required so Codex, Claude Code, and human developers can implement and test the module incrementally without hidden coupling.

User-output note: any user-visible output related to AQELYN Knowledge Graph shall be written in clear language first and detailed technical language second. The system must always answer: what was found, why it matters, how AQELYN knows, and what the user can do next.

Government and enterprise note: AQELYN Knowledge Graph shall support auditability, exportable records, retention policy alignment, and review-ready evidence packages. The design supports private users, companies, managed service providers, and government environments without creating separate architectures.

006 - Stakeholders and Users

Stakeholders and Users for AQELYN Knowledge Graph shall be implemented in accordance with AQELYN Platform Architecture Baseline v1.0. The implementation must preserve traceability, deterministic behavior, explainability, and evidence-backed outputs.

This section defines engineering expectations for knowledge graph and provides the implementation team with stable guidance that should not be bypassed by local optimization or AI-generated redesign.

Implementation note: AQELYN Knowledge Graph must treat every decision as auditable. If the engine produces a finding, state transition, recommendation, event, or policy result, that output must reference the reason, source, timestamp, and confidence context. This supports AQELYN's user promise that security outcomes are understandable to non-experts and actionable by experts.

Quality note: AQELYN Knowledge Graph shall include code-level tests, schema-level tests, and documentation-level traceability. Any deviation discovered during implementation shall be raised as an Engineering Change Request rather than silently changing the architecture.

Operational note: AQELYN Knowledge Graph shall expose health state, readiness state, dependency state, and degraded-mode state. Operators must be able to understand whether the engine is fully operational, partially degraded, blocked by an upstream dependency, or intentionally disabled by policy.

Developer note: implementation of AQELYN Knowledge Graph shall follow clean boundaries between domain logic, infrastructure adapters, API handlers, persistence adapters, and event adapters. This is required so Codex, Claude Code, and human developers can implement and test the module incrementally without hidden coupling.

User-output note: any user-visible output related to AQELYN Knowledge Graph shall be written in clear language first and detailed technical language second. The system must always answer: what was found, why it matters, how AQELYN knows, and what the user can do next.

Government and enterprise note: AQELYN Knowledge Graph shall support auditability, exportable records, retention policy alignment, and review-ready evidence packages. The design supports private users, companies, managed service providers, and government environments without creating separate architectures.

007 - Functional Requirements

- ****FR-001:**** AQELYN Knowledge Graph shall provide capability 1 for knowledge graph with deterministic results, structured outputs, and traceable evidence references.

- ****FR-002:**** AQELYN Knowledge Graph shall provide capability 2 for knowledge graph with deterministic results, structured outputs, and traceable evidence references.
- ****FR-003:**** AQELYN Knowledge Graph shall provide capability 3 for knowledge graph with deterministic results, structured outputs, and traceable evidence references.
- ****FR-004:**** AQELYN Knowledge Graph shall provide capability 4 for knowledge graph with deterministic results, structured outputs, and traceable evidence references.
- ****FR-005:**** AQELYN Knowledge Graph shall provide capability 5 for knowledge graph with deterministic results, structured outputs, and traceable evidence references.
- ****FR-006:**** AQELYN Knowledge Graph shall provide capability 6 for knowledge graph with deterministic results, structured outputs, and traceable evidence references.
- ****FR-007:**** AQELYN Knowledge Graph shall provide capability 7 for knowledge graph with deterministic results, structured outputs, and traceable evidence references.
- ****FR-008:**** AQELYN Knowledge Graph shall provide capability 8 for knowledge graph with deterministic results, structured outputs, and traceable evidence references.
- ****FR-009:**** AQELYN Knowledge Graph shall provide capability 9 for knowledge graph with deterministic results, structured outputs, and traceable evidence references.
- ****FR-010:**** AQELYN Knowledge Graph shall provide capability 10 for knowledge graph with deterministic results, structured outputs, and traceable evidence references.

Implementation note: AQELYN Knowledge Graph must treat every decision as auditable. If the engine produces a finding, state transition, recommendation, event, or policy result, that output must reference the reason, source, timestamp, and confidence context. This supports AQELYN's user promise that security outcomes are understandable to non-experts and actionable by experts.

Quality note: AQELYN Knowledge Graph shall include code-level tests, schema-level tests, and documentation-level traceability. Any deviation discovered during implementation shall be raised as an Engineering Change Request rather than silently changing the architecture.

Operational note: AQELYN Knowledge Graph shall expose health state, readiness state, dependency state, and degraded-mode state. Operators must be able to understand whether the engine is fully operational, partially degraded, blocked by an upstream dependency, or intentionally disabled by policy.

Developer note: implementation of AQELYN Knowledge Graph shall follow clean boundaries between domain logic, infrastructure adapters, API handlers, persistence adapters, and event adapters. This is required so Codex, Claude Code, and human developers can implement and test the module incrementally without hidden coupling.

User-output note: any user-visible output related to AQELYN Knowledge Graph shall be written in clear language first and detailed technical language second. The system must always answer: what was found, why it matters, how AQELYN knows, and what the user can do next.

Government and enterprise note: AQELYN Knowledge Graph shall support auditability, exportable records, retention policy alignment, and review-ready evidence packages. The design supports private users, companies, managed service providers, and government environments without creating separate architectures.

008 - Non-Functional Requirements

- Availability target: the module shall support resilient operation under nominal platform load.
- Performance target: processing shall be measurable, observable, and suitable for incremental scale-out.
- Maintainability target: code shall follow EA-0058 coding standards and EA-0061 developer workflow.
- Usability target: all end-user messages shall be understandable by non-experts while preserving expert detail on demand.

Implementation note: AQELYN Knowledge Graph must treat every decision as auditable. If the engine produces a finding, state transition, recommendation, event, or policy result, that output must reference the reason, source, timestamp, and

confidence context. This supports AQELYN's user promise that security outcomes are understandable to non-experts and actionable by experts.

Quality note: AQELYN Knowledge Graph shall include code-level tests, schema-level tests, and documentation-level traceability. Any deviation discovered during implementation shall be raised as an Engineering Change Request rather than silently changing the architecture.

Operational note: AQELYN Knowledge Graph shall expose health state, readiness state, dependency state, and degraded-mode state. Operators must be able to understand whether the engine is fully operational, partially degraded, blocked by an upstream dependency, or intentionally disabled by policy.

Developer note: implementation of AQELYN Knowledge Graph shall follow clean boundaries between domain logic, infrastructure adapters, API handlers, persistence adapters, and event adapters. This is required so Codex, Claude Code, and human developers can implement and test the module incrementally without hidden coupling.

User-output note: any user-visible output related to AQELYN Knowledge Graph shall be written in clear language first and detailed technical language second. The system must always answer: what was found, why it matters, how AQELYN knows, and what the user can do next.

Government and enterprise note: AQELYN Knowledge Graph shall support auditability, exportable records, retention policy alignment, and review-ready evidence packages. The design supports private users, companies, managed service providers, and government environments without creating separate architectures.

009 - Security Requirements

- All service-to-service communication shall be authenticated and authorized.
- Secrets shall never be hard-coded or logged.
- Inputs shall be validated at every trust boundary.
- Security-relevant decisions shall generate audit records.
- Events and evidence references shall be tamper-evident where required by the Evidence Engine.

Implementation note: AQELYN Knowledge Graph must treat every decision as auditable. If the engine produces a finding, state transition, recommendation, event, or policy result, that output must reference the reason, source, timestamp, and confidence context. This supports AQELYN's user promise that security outcomes are understandable to non-experts and actionable by experts.

Quality note: AQELYN Knowledge Graph shall include code-level tests, schema-level tests, and documentation-level traceability. Any deviation discovered during implementation shall be raised as an Engineering Change Request rather than silently changing the architecture.

Operational note: AQELYN Knowledge Graph shall expose health state, readiness state, dependency state, and degraded-mode state. Operators must be able to understand whether the engine is fully operational, partially degraded, blocked by an upstream dependency, or intentionally disabled by policy.

Developer note: implementation of AQELYN Knowledge Graph shall follow clean boundaries between domain logic, infrastructure adapters, API handlers, persistence adapters, and event adapters. This is required so Codex, Claude Code, and human developers can implement and test the module incrementally without hidden coupling.

User-output note: any user-visible output related to AQELYN Knowledge Graph shall be written in clear language first and detailed technical language second. The system must always answer: what was found, why it matters, how AQELYN knows, and what the user can do next.

Government and enterprise note: AQELYN Knowledge Graph shall support auditability, exportable records, retention policy alignment, and review-ready evidence packages. The design supports private users, companies, managed service providers, and government environments without creating separate architectures.

010 - Privacy Requirements

Privacy Requirements for AQELYN Knowledge Graph shall be implemented in accordance with AQELYN Platform Architecture Baseline v1.0. The implementation must preserve traceability, deterministic behavior, explainability, and evidence-backed outputs.

This section defines engineering expectations for knowledge graph and provides the implementation team with stable guidance that should not be bypassed by local optimization or AI-generated redesign.

Implementation note: AQELYN Knowledge Graph must treat every decision as auditable. If the engine produces a finding, state transition, recommendation, event, or policy result, that output must reference the reason, source, timestamp, and confidence context. This supports AQELYN's user promise that security outcomes are understandable to non-experts and actionable by experts.

Quality note: AQELYN Knowledge Graph shall include code-level tests, schema-level tests, and documentation-level traceability. Any deviation discovered during implementation shall be raised as an Engineering Change Request rather than silently changing the architecture.

Operational note: AQELYN Knowledge Graph shall expose health state, readiness state, dependency state, and degraded-mode state. Operators must be able to understand whether the engine is fully operational, partially degraded, blocked by an upstream dependency, or intentionally disabled by policy.

Developer note: implementation of AQELYN Knowledge Graph shall follow clean boundaries between domain logic, infrastructure adapters, API handlers, persistence adapters, and event adapters. This is required so Codex, Claude Code, and human developers can implement and test the module incrementally without hidden coupling.

User-output note: any user-visible output related to AQELYN Knowledge Graph shall be written in clear language first and detailed technical language second. The system must always answer: what was found, why it matters, how AQELYN knows, and what the user can do next.

Government and enterprise note: AQELYN Knowledge Graph shall support auditability, exportable records, retention policy alignment, and review-ready evidence packages. The design supports private users, companies, managed service providers, and government environments without creating separate architectures.

011 - System Context

System Context for AQELYN Knowledge Graph shall be implemented in accordance with AQELYN Platform Architecture Baseline v1.0. The implementation must preserve traceability, deterministic behavior, explainability, and evidence-backed outputs.

This section defines engineering expectations for knowledge graph and provides the implementation team with stable guidance that should not be bypassed by local optimization or AI-generated redesign.

Implementation note: AQELYN Knowledge Graph must treat every decision as auditable. If the engine produces a finding, state transition, recommendation, event, or policy result, that output must reference the reason, source, timestamp, and confidence context. This supports AQELYN's user promise that security outcomes are understandable to non-experts and actionable by experts.

Quality note: AQELYN Knowledge Graph shall include code-level tests, schema-level tests, and documentation-level traceability. Any deviation discovered during implementation shall be raised as an Engineering Change Request rather than silently changing the architecture.

Operational note: AQELYN Knowledge Graph shall expose health state, readiness state, dependency state, and degraded-mode state. Operators must be able to understand whether the engine is fully operational, partially degraded, blocked by an upstream dependency, or intentionally disabled by policy.

Developer note: implementation of AQELYN Knowledge Graph shall follow clean boundaries between domain logic, infrastructure adapters, API handlers, persistence adapters, and event adapters. This is required so Codex, Claude Code, and human developers can implement and test the module incrementally without hidden coupling.

User-output note: any user-visible output related to AQELYN Knowledge Graph shall be written in clear language first and detailed technical language second. The system must always answer: what was found, why it matters, how AQELYN knows, and what the user can do next.

Government and enterprise note: AQELYN Knowledge Graph shall support auditability, exportable records, retention policy alignment, and review-ready evidence packages. The design supports private users, companies, managed service providers, and government environments without creating separate architectures.

012 - Architecture Overview

AQELYN Knowledge Graph follows the AQELYN modular engine architecture. It exposes versioned interfaces, consumes and publishes events, stores only approved canonical objects, and integrates through the Event Bus rather than direct hidden coupling.

```
Inputs -> AQELYN Knowledge Graph -> Canonical Objects -> Events -> Evidence -> Knowledge Graph -> Recommendations
```

Implementation note: AQELYN Knowledge Graph must treat every decision as auditable. If the engine produces a finding, state transition, recommendation, event, or policy result, that output must reference the reason, source, timestamp, and confidence context. This supports AQELYN's user promise that security outcomes are understandable to non-experts and actionable by experts.

Quality note: AQELYN Knowledge Graph shall include code-level tests, schema-level tests, and documentation-level traceability. Any deviation discovered during implementation shall be raised as an Engineering Change Request rather than silently changing the architecture.

Operational note: AQELYN Knowledge Graph shall expose health state, readiness state, dependency state, and degraded-mode state. Operators must be able to understand whether the engine is fully operational, partially degraded, blocked by an upstream dependency, or intentionally disabled by policy.

Developer note: implementation of AQELYN Knowledge Graph shall follow clean boundaries between domain logic, infrastructure adapters, API handlers, persistence adapters, and event adapters. This is required so Codex, Claude Code, and human developers can implement and test the module incrementally without hidden coupling.

User-output note: any user-visible output related to AQELYN Knowledge Graph shall be written in clear language first and detailed technical language second. The system must always answer: what was found, why it matters, how AQELYN knows, and what the user can do next.

Government and enterprise note: AQELYN Knowledge Graph shall support auditability, exportable records, retention policy alignment, and review-ready evidence packages. The design supports private users, companies, managed service providers, and government environments without creating separate architectures.

013 - Component Architecture

- ****Controller:**** Provides a bounded implementation responsibility within AQELYN Knowledge Graph and must expose testable interfaces.
- ****Service Layer:**** Provides a bounded implementation responsibility within AQELYN Knowledge Graph and must expose testable interfaces.
- ****Repository Adapter:**** Provides a bounded implementation responsibility within AQELYN Knowledge Graph and must expose testable interfaces.
- ****Event Adapter:**** Provides a bounded implementation responsibility within AQELYN Knowledge Graph and must expose testable interfaces.
- ****Evidence Adapter:**** Provides a bounded implementation responsibility within AQELYN Knowledge Graph and must expose testable interfaces.
- ****Policy Adapter:**** Provides a bounded implementation responsibility within AQELYN Knowledge Graph and must expose testable interfaces.

- **Telemetry Adapter:** Provides a bounded implementation responsibility within AQELYN Knowledge Graph and must expose testable interfaces.
- **Validation Layer:** Provides a bounded implementation responsibility within AQELYN Knowledge Graph and must expose testable interfaces.

Implementation note: AQELYN Knowledge Graph must treat every decision as auditable. If the engine produces a finding, state transition, recommendation, event, or policy result, that output must reference the reason, source, timestamp, and confidence context. This supports AQELYN's user promise that security outcomes are understandable to non-experts and actionable by experts.

Quality note: AQELYN Knowledge Graph shall include code-level tests, schema-level tests, and documentation-level traceability. Any deviation discovered during implementation shall be raised as an Engineering Change Request rather than silently changing the architecture.

Operational note: AQELYN Knowledge Graph shall expose health state, readiness state, dependency state, and degraded-mode state. Operators must be able to understand whether the engine is fully operational, partially degraded, blocked by an upstream dependency, or intentionally disabled by policy.

Developer note: implementation of AQELYN Knowledge Graph shall follow clean boundaries between domain logic, infrastructure adapters, API handlers, persistence adapters, and event adapters. This is required so Codex, Claude Code, and human developers can implement and test the module incrementally without hidden coupling.

User-output note: any user-visible output related to AQELYN Knowledge Graph shall be written in clear language first and detailed technical language second. The system must always answer: what was found, why it matters, how AQELYN knows, and what the user can do next.

Government and enterprise note: AQELYN Knowledge Graph shall support auditability, exportable records, retention policy alignment, and review-ready evidence packages. The design supports private users, companies, managed service providers, and government environments without creating separate architectures.

014 - Data Model

AQELYN Knowledge Graph shall use the Universal Object Model and shall not introduce isolated object representations. Each persisted object must include identifiers, timestamps, provenance, version, source, trust context, and evidence references where applicable.

| Object | Purpose |

|---|---|

| Knowledge GraphRecord | Canonical state representation |

| Knowledge GraphEvent | Event Bus message payload |

| Knowledge GraphEvidence | Evidence reference container |

| Knowledge GraphAssessment | Assessment and decision output |

Implementation note: AQELYN Knowledge Graph must treat every decision as auditable. If the engine produces a finding, state transition, recommendation, event, or policy result, that output must reference the reason, source, timestamp, and confidence context. This supports AQELYN's user promise that security outcomes are understandable to non-experts and actionable by experts.

Quality note: AQELYN Knowledge Graph shall include code-level tests, schema-level tests, and documentation-level traceability. Any deviation discovered during implementation shall be raised as an Engineering Change Request rather than silently changing the architecture.

Operational note: AQELYN Knowledge Graph shall expose health state, readiness state, dependency state, and degraded-mode state. Operators must be able to understand whether the engine is fully operational, partially degraded, blocked by an upstream dependency, or intentionally disabled by policy.

Developer note: implementation of AQELYN Knowledge Graph shall follow clean boundaries between domain logic, infrastructure adapters, API handlers, persistence adapters, and event adapters. This is required so Codex, Claude Code, and human developers can implement and test the module incrementally without hidden coupling.

User-output note: any user-visible output related to AQELYN Knowledge Graph shall be written in clear language first and detailed technical language second. The system must always answer: what was found, why it matters, how AQELYN knows, and what the user can do next.

Government and enterprise note: AQELYN Knowledge Graph shall support auditability, exportable records, retention policy alignment, and review-ready evidence packages. The design supports private users, companies, managed service providers, and government environments without creating separate architectures.

015 - Event Model

- **KnowledgeGraphCreated**: emitted using the AQELYN universal event envelope.
- **KnowledgeGraphUpdated**: emitted using the AQELYN universal event envelope.
- **KnowledgeGraphValidated**: emitted using the AQELYN universal event envelope.
- **KnowledgeGraphAssessmentCompleted**: emitted using the AQELYN universal event envelope.
- **KnowledgeGraphPolicyEvaluated**: emitted using the AQELYN universal event envelope.
- **KnowledgeGraphEvidenceLinked**: emitted using the AQELYN universal event envelope.
- **KnowledgeGraphHealthChanged**: emitted using the AQELYN universal event envelope.
- **KnowledgeGraphErrorRaised**: emitted using the AQELYN universal event envelope.

Implementation note: AQELYN Knowledge Graph must treat every decision as auditable. If the engine produces a finding, state transition, recommendation, event, or policy result, that output must reference the reason, source, timestamp, and confidence context. This supports AQELYN's user promise that security outcomes are understandable to non-experts and actionable by experts.

Quality note: AQELYN Knowledge Graph shall include code-level tests, schema-level tests, and documentation-level traceability. Any deviation discovered during implementation shall be raised as an Engineering Change Request rather than silently changing the architecture.

Operational note: AQELYN Knowledge Graph shall expose health state, readiness state, dependency state, and degraded-mode state. Operators must be able to understand whether the engine is fully operational, partially degraded, blocked by an upstream dependency, or intentionally disabled by policy.

Developer note: implementation of AQELYN Knowledge Graph shall follow clean boundaries between domain logic, infrastructure adapters, API handlers, persistence adapters, and event adapters. This is required so Codex, Claude Code, and human developers can implement and test the module incrementally without hidden coupling.

User-output note: any user-visible output related to AQELYN Knowledge Graph shall be written in clear language first and detailed technical language second. The system must always answer: what was found, why it matters, how AQELYN knows, and what the user can do next.

Government and enterprise note: AQELYN Knowledge Graph shall support auditability, exportable records, retention policy alignment, and review-ready evidence packages. The design supports private users, companies, managed service providers, and government environments without creating separate architectures.

016 - API and Interface Contracts

All interfaces shall be versioned, documented, validated, and tested. Internal APIs shall use predictable naming, typed schemas, stable error envelopes, and correlation identifiers.

```
GET /api/v1/knowledge-graph
POST /api/v1/knowledge-graph/evaluate
GET /api/v1/knowledge-graph/health
```

Implementation note: AQELYN Knowledge Graph must treat every decision as auditable. If the engine produces a finding, state transition, recommendation, event, or policy result, that output must reference the reason, source, timestamp, and confidence context. This supports AQELYN's user promise that security outcomes are understandable to non-experts and actionable by experts.

Quality note: AQELYN Knowledge Graph shall include code-level tests, schema-level tests, and documentation-level traceability. Any deviation discovered during implementation shall be raised as an Engineering Change Request rather than silently changing the architecture.

Operational note: AQELYN Knowledge Graph shall expose health state, readiness state, dependency state, and degraded-mode state. Operators must be able to understand whether the engine is fully operational, partially degraded, blocked by an upstream dependency, or intentionally disabled by policy.

Developer note: implementation of AQELYN Knowledge Graph shall follow clean boundaries between domain logic, infrastructure adapters, API handlers, persistence adapters, and event adapters. This is required so Codex, Claude Code, and human developers can implement and test the module incrementally without hidden coupling.

User-output note: any user-visible output related to AQELYN Knowledge Graph shall be written in clear language first and detailed technical language second. The system must always answer: what was found, why it matters, how AQELYN knows, and what the user can do next.

Government and enterprise note: AQELYN Knowledge Graph shall support auditability, exportable records, retention policy alignment, and review-ready evidence packages. The design supports private users, companies, managed service providers, and government environments without creating separate architectures.

017 - Integration Model

This module integrates with the following platform capabilities where applicable:

- AQELYN Kernel for lifecycle management.
- Universal Object Model for canonical data.
- Event Bus for asynchronous communication.
- Evidence Engine for traceable facts.
- Knowledge Graph for relationship context.
- Trust, Policy, Workflow, and Mission engines for decisions and remediation.

Implementation note: AQELYN Knowledge Graph must treat every decision as auditable. If the engine produces a finding, state transition, recommendation, event, or policy result, that output must reference the reason, source, timestamp, and confidence context. This supports AQELYN's user promise that security outcomes are understandable to non-experts and actionable by experts.

Quality note: AQELYN Knowledge Graph shall include code-level tests, schema-level tests, and documentation-level traceability. Any deviation discovered during implementation shall be raised as an Engineering Change Request rather than silently changing the architecture.

Operational note: AQELYN Knowledge Graph shall expose health state, readiness state, dependency state, and degraded-mode state. Operators must be able to understand whether the engine is fully operational, partially degraded, blocked by an upstream dependency, or intentionally disabled by policy.

Developer note: implementation of AQELYN Knowledge Graph shall follow clean boundaries between domain logic, infrastructure adapters, API handlers, persistence adapters, and event adapters. This is required so Codex, Claude Code, and human developers can implement and test the module incrementally without hidden coupling.

User-output note: any user-visible output related to AQELYN Knowledge Graph shall be written in clear language first and detailed technical language second. The system must always answer: what was found, why it matters, how AQELYN knows, and what the user can do next.

Government and enterprise note: AQELYN Knowledge Graph shall support auditability, exportable records, retention policy alignment, and review-ready evidence packages. The design supports private users, companies, managed service providers, and government environments without creating separate architectures.

018 - Operational Lifecycle

Initialize -> Configure -> Validate -> Start -> Process -> Observe -> Degrade Safely -> Stop -> Archive State

Every lifecycle transition shall be logged and testable.

Implementation note: AQELYN Knowledge Graph must treat every decision as auditable. If the engine produces a finding, state transition, recommendation, event, or policy result, that output must reference the reason, source, timestamp, and confidence context. This supports AQELYN's user promise that security outcomes are understandable to non-experts and actionable by experts.

Quality note: AQELYN Knowledge Graph shall include code-level tests, schema-level tests, and documentation-level traceability. Any deviation discovered during implementation shall be raised as an Engineering Change Request rather than silently changing the architecture.

Operational note: AQELYN Knowledge Graph shall expose health state, readiness state, dependency state, and degraded-mode state. Operators must be able to understand whether the engine is fully operational, partially degraded, blocked by an upstream dependency, or intentionally disabled by policy.

Developer note: implementation of AQELYN Knowledge Graph shall follow clean boundaries between domain logic, infrastructure adapters, API handlers, persistence adapters, and event adapters. This is required so Codex, Claude Code, and human developers can implement and test the module incrementally without hidden coupling.

User-output note: any user-visible output related to AQELYN Knowledge Graph shall be written in clear language first and detailed technical language second. The system must always answer: what was found, why it matters, how AQELYN knows, and what the user can do next.

Government and enterprise note: AQELYN Knowledge Graph shall support auditability, exportable records, retention policy alignment, and review-ready evidence packages. The design supports private users, companies, managed service providers, and government environments without creating separate architectures.

019 - Configuration Model

Configuration Model for AQELYN Knowledge Graph shall be implemented in accordance with AQELYN Platform Architecture Baseline v1.0. The implementation must preserve traceability, deterministic behavior, explainability, and evidence-backed outputs.

This section defines engineering expectations for knowledge graph and provides the implementation team with stable guidance that should not be bypassed by local optimization or AI-generated redesign.

Implementation note: AQELYN Knowledge Graph must treat every decision as auditable. If the engine produces a finding, state transition, recommendation, event, or policy result, that output must reference the reason, source, timestamp, and confidence context. This supports AQELYN's user promise that security outcomes are understandable to non-experts and actionable by experts.

Quality note: AQELYN Knowledge Graph shall include code-level tests, schema-level tests, and documentation-level traceability. Any deviation discovered during implementation shall be raised as an Engineering Change Request rather than silently changing the architecture.

Operational note: AQELYN Knowledge Graph shall expose health state, readiness state, dependency state, and degraded-mode state. Operators must be able to understand whether the engine is fully operational, partially degraded, blocked by an upstream dependency, or intentionally disabled by policy.

Developer note: implementation of AQELYN Knowledge Graph shall follow clean boundaries between domain logic, infrastructure adapters, API handlers, persistence adapters, and event adapters. This is required so Codex, Claude Code, and human developers can implement and test the module incrementally without hidden coupling.

User-output note: any user-visible output related to AQELYN Knowledge Graph shall be written in clear language first and detailed technical language second. The system must always answer: what was found, why it matters, how AQELYN knows, and what the user can do next.

Government and enterprise note: AQELYN Knowledge Graph shall support auditability, exportable records, retention policy alignment, and review-ready evidence packages. The design supports private users, companies, managed service providers, and government environments without creating separate architectures.

020 - Error Handling

Error Handling for AQELYN Knowledge Graph shall be implemented in accordance with AQELYN Platform Architecture Baseline v1.0. The implementation must preserve traceability, deterministic behavior, explainability, and evidence-backed outputs.

This section defines engineering expectations for knowledge graph and provides the implementation team with stable guidance that should not be bypassed by local optimization or AI-generated redesign.

Implementation note: AQELYN Knowledge Graph must treat every decision as auditable. If the engine produces a finding, state transition, recommendation, event, or policy result, that output must reference the reason, source, timestamp, and confidence context. This supports AQELYN's user promise that security outcomes are understandable to non-experts and actionable by experts.

Quality note: AQELYN Knowledge Graph shall include code-level tests, schema-level tests, and documentation-level traceability. Any deviation discovered during implementation shall be raised as an Engineering Change Request rather than silently changing the architecture.

Operational note: AQELYN Knowledge Graph shall expose health state, readiness state, dependency state, and degraded-mode state. Operators must be able to understand whether the engine is fully operational, partially degraded, blocked by an upstream dependency, or intentionally disabled by policy.

Developer note: implementation of AQELYN Knowledge Graph shall follow clean boundaries between domain logic, infrastructure adapters, API handlers, persistence adapters, and event adapters. This is required so Codex, Claude Code, and human developers can implement and test the module incrementally without hidden coupling.

User-output note: any user-visible output related to AQELYN Knowledge Graph shall be written in clear language first and detailed technical language second. The system must always answer: what was found, why it matters, how AQELYN knows, and what the user can do next.

Government and enterprise note: AQELYN Knowledge Graph shall support auditability, exportable records, retention policy alignment, and review-ready evidence packages. The design supports private users, companies, managed service providers, and government environments without creating separate architectures.

021 - Observability

Observability for AQELYN Knowledge Graph shall be implemented in accordance with AQELYN Platform Architecture Baseline v1.0. The implementation must preserve traceability, deterministic behavior, explainability, and evidence-backed outputs.

This section defines engineering expectations for knowledge graph and provides the implementation team with stable guidance that should not be bypassed by local optimization or AI-generated redesign.

Implementation note: AQELYN Knowledge Graph must treat every decision as auditable. If the engine produces a finding, state transition, recommendation, event, or policy result, that output must reference the reason, source, timestamp, and confidence context. This supports AQELYN's user promise that security outcomes are understandable to non-experts and

actionable by experts.

Quality note: AQELYN Knowledge Graph shall include code-level tests, schema-level tests, and documentation-level traceability. Any deviation discovered during implementation shall be raised as an Engineering Change Request rather than silently changing the architecture.

Operational note: AQELYN Knowledge Graph shall expose health state, readiness state, dependency state, and degraded-mode state. Operators must be able to understand whether the engine is fully operational, partially degraded, blocked by an upstream dependency, or intentionally disabled by policy.

Developer note: implementation of AQELYN Knowledge Graph shall follow clean boundaries between domain logic, infrastructure adapters, API handlers, persistence adapters, and event adapters. This is required so Codex, Claude Code, and human developers can implement and test the module incrementally without hidden coupling.

User-output note: any user-visible output related to AQELYN Knowledge Graph shall be written in clear language first and detailed technical language second. The system must always answer: what was found, why it matters, how AQELYN knows, and what the user can do next.

Government and enterprise note: AQELYN Knowledge Graph shall support auditability, exportable records, retention policy alignment, and review-ready evidence packages. The design supports private users, companies, managed service providers, and government environments without creating separate architectures.

022 - Logging

Logging for AQELYN Knowledge Graph shall be implemented in accordance with AQELYN Platform Architecture Baseline v1.0. The implementation must preserve traceability, deterministic behavior, explainability, and evidence-backed outputs.

This section defines engineering expectations for knowledge graph and provides the implementation team with stable guidance that should not be bypassed by local optimization or AI-generated redesign.

Implementation note: AQELYN Knowledge Graph must treat every decision as auditable. If the engine produces a finding, state transition, recommendation, event, or policy result, that output must reference the reason, source, timestamp, and confidence context. This supports AQELYN's user promise that security outcomes are understandable to non-experts and actionable by experts.

Quality note: AQELYN Knowledge Graph shall include code-level tests, schema-level tests, and documentation-level traceability. Any deviation discovered during implementation shall be raised as an Engineering Change Request rather than silently changing the architecture.

Operational note: AQELYN Knowledge Graph shall expose health state, readiness state, dependency state, and degraded-mode state. Operators must be able to understand whether the engine is fully operational, partially degraded, blocked by an upstream dependency, or intentionally disabled by policy.

Developer note: implementation of AQELYN Knowledge Graph shall follow clean boundaries between domain logic, infrastructure adapters, API handlers, persistence adapters, and event adapters. This is required so Codex, Claude Code, and human developers can implement and test the module incrementally without hidden coupling.

User-output note: any user-visible output related to AQELYN Knowledge Graph shall be written in clear language first and detailed technical language second. The system must always answer: what was found, why it matters, how AQELYN knows, and what the user can do next.

Government and enterprise note: AQELYN Knowledge Graph shall support auditability, exportable records, retention policy alignment, and review-ready evidence packages. The design supports private users, companies, managed service providers, and government environments without creating separate architectures.

023 - Metrics

Metrics for AQELYN Knowledge Graph shall be implemented in accordance with AQELYN Platform Architecture Baseline v1.0. The implementation must preserve traceability, deterministic behavior, explainability, and evidence-backed outputs.

This section defines engineering expectations for knowledge graph and provides the implementation team with stable guidance that should not be bypassed by local optimization or AI-generated redesign.

Implementation note: AQELYN Knowledge Graph must treat every decision as auditable. If the engine produces a finding, state transition, recommendation, event, or policy result, that output must reference the reason, source, timestamp, and confidence context. This supports AQELYN's user promise that security outcomes are understandable to non-experts and actionable by experts.

Quality note: AQELYN Knowledge Graph shall include code-level tests, schema-level tests, and documentation-level traceability. Any deviation discovered during implementation shall be raised as an Engineering Change Request rather than silently changing the architecture.

Operational note: AQELYN Knowledge Graph shall expose health state, readiness state, dependency state, and degraded-mode state. Operators must be able to understand whether the engine is fully operational, partially degraded, blocked by an upstream dependency, or intentionally disabled by policy.

Developer note: implementation of AQELYN Knowledge Graph shall follow clean boundaries between domain logic, infrastructure adapters, API handlers, persistence adapters, and event adapters. This is required so Codex, Claude Code, and human developers can implement and test the module incrementally without hidden coupling.

User-output note: any user-visible output related to AQELYN Knowledge Graph shall be written in clear language first and detailed technical language second. The system must always answer: what was found, why it matters, how AQELYN knows, and what the user can do next.

Government and enterprise note: AQELYN Knowledge Graph shall support auditability, exportable records, retention policy alignment, and review-ready evidence packages. The design supports private users, companies, managed service providers, and government environments without creating separate architectures.

024 - Audit and Evidence

Audit and Evidence for AQELYN Knowledge Graph shall be implemented in accordance with AQELYN Platform Architecture Baseline v1.0. The implementation must preserve traceability, deterministic behavior, explainability, and evidence-backed outputs.

This section defines engineering expectations for knowledge graph and provides the implementation team with stable guidance that should not be bypassed by local optimization or AI-generated redesign.

Implementation note: AQELYN Knowledge Graph must treat every decision as auditable. If the engine produces a finding, state transition, recommendation, event, or policy result, that output must reference the reason, source, timestamp, and confidence context. This supports AQELYN's user promise that security outcomes are understandable to non-experts and actionable by experts.

Quality note: AQELYN Knowledge Graph shall include code-level tests, schema-level tests, and documentation-level traceability. Any deviation discovered during implementation shall be raised as an Engineering Change Request rather than silently changing the architecture.

Operational note: AQELYN Knowledge Graph shall expose health state, readiness state, dependency state, and degraded-mode state. Operators must be able to understand whether the engine is fully operational, partially degraded, blocked by an upstream dependency, or intentionally disabled by policy.

Developer note: implementation of AQELYN Knowledge Graph shall follow clean boundaries between domain logic, infrastructure adapters, API handlers, persistence adapters, and event adapters. This is required so Codex, Claude Code, and human developers can implement and test the module incrementally without hidden coupling.

User-output note: any user-visible output related to AQELYN Knowledge Graph shall be written in clear language first and detailed technical language second. The system must always answer: what was found, why it matters, how AQELYN knows, and what the user can do next.

Government and enterprise note: AQELYN Knowledge Graph shall support auditability, exportable records, retention policy alignment, and review-ready evidence packages. The design supports private users, companies, managed service providers, and government environments without creating separate architectures.

025 - Trust and Confidence

Trust and Confidence for AQELYN Knowledge Graph shall be implemented in accordance with AQELYN Platform Architecture Baseline v1.0. The implementation must preserve traceability, deterministic behavior, explainability, and evidence-backed outputs.

This section defines engineering expectations for knowledge graph and provides the implementation team with stable guidance that should not be bypassed by local optimization or AI-generated redesign.

Implementation note: AQELYN Knowledge Graph must treat every decision as auditable. If the engine produces a finding, state transition, recommendation, event, or policy result, that output must reference the reason, source, timestamp, and confidence context. This supports AQELYN's user promise that security outcomes are understandable to non-experts and actionable by experts.

Quality note: AQELYN Knowledge Graph shall include code-level tests, schema-level tests, and documentation-level traceability. Any deviation discovered during implementation shall be raised as an Engineering Change Request rather than silently changing the architecture.

Operational note: AQELYN Knowledge Graph shall expose health state, readiness state, dependency state, and degraded-mode state. Operators must be able to understand whether the engine is fully operational, partially degraded, blocked by an upstream dependency, or intentionally disabled by policy.

Developer note: implementation of AQELYN Knowledge Graph shall follow clean boundaries between domain logic, infrastructure adapters, API handlers, persistence adapters, and event adapters. This is required so Codex, Claude Code, and human developers can implement and test the module incrementally without hidden coupling.

User-output note: any user-visible output related to AQELYN Knowledge Graph shall be written in clear language first and detailed technical language second. The system must always answer: what was found, why it matters, how AQELYN knows, and what the user can do next.

Government and enterprise note: AQELYN Knowledge Graph shall support auditability, exportable records, retention policy alignment, and review-ready evidence packages. The design supports private users, companies, managed service providers, and government environments without creating separate architectures.

026 - Policy and Governance

Policy and Governance for AQELYN Knowledge Graph shall be implemented in accordance with AQELYN Platform Architecture Baseline v1.0. The implementation must preserve traceability, deterministic behavior, explainability, and evidence-backed outputs.

This section defines engineering expectations for knowledge graph and provides the implementation team with stable guidance that should not be bypassed by local optimization or AI-generated redesign.

Implementation note: AQELYN Knowledge Graph must treat every decision as auditable. If the engine produces a finding, state transition, recommendation, event, or policy result, that output must reference the reason, source, timestamp, and confidence context. This supports AQELYN's user promise that security outcomes are understandable to non-experts and actionable by experts.

Quality note: AQELYN Knowledge Graph shall include code-level tests, schema-level tests, and documentation-level traceability. Any deviation discovered during implementation shall be raised as an Engineering Change Request rather

than silently changing the architecture.

Operational note: AQELYN Knowledge Graph shall expose health state, readiness state, dependency state, and degraded-mode state. Operators must be able to understand whether the engine is fully operational, partially degraded, blocked by an upstream dependency, or intentionally disabled by policy.

Developer note: implementation of AQELYN Knowledge Graph shall follow clean boundaries between domain logic, infrastructure adapters, API handlers, persistence adapters, and event adapters. This is required so Codex, Claude Code, and human developers can implement and test the module incrementally without hidden coupling.

User-output note: any user-visible output related to AQELYN Knowledge Graph shall be written in clear language first and detailed technical language second. The system must always answer: what was found, why it matters, how AQELYN knows, and what the user can do next.

Government and enterprise note: AQELYN Knowledge Graph shall support auditability, exportable records, retention policy alignment, and review-ready evidence packages. The design supports private users, companies, managed service providers, and government environments without creating separate architectures.

027 - Performance and Scalability

Performance and Scalability for AQELYN Knowledge Graph shall be implemented in accordance with AQELYN Platform Architecture Baseline v1.0. The implementation must preserve traceability, deterministic behavior, explainability, and evidence-backed outputs.

This section defines engineering expectations for knowledge graph and provides the implementation team with stable guidance that should not be bypassed by local optimization or AI-generated redesign.

Implementation note: AQELYN Knowledge Graph must treat every decision as auditable. If the engine produces a finding, state transition, recommendation, event, or policy result, that output must reference the reason, source, timestamp, and confidence context. This supports AQELYN's user promise that security outcomes are understandable to non-experts and actionable by experts.

Quality note: AQELYN Knowledge Graph shall include code-level tests, schema-level tests, and documentation-level traceability. Any deviation discovered during implementation shall be raised as an Engineering Change Request rather than silently changing the architecture.

Operational note: AQELYN Knowledge Graph shall expose health state, readiness state, dependency state, and degraded-mode state. Operators must be able to understand whether the engine is fully operational, partially degraded, blocked by an upstream dependency, or intentionally disabled by policy.

Developer note: implementation of AQELYN Knowledge Graph shall follow clean boundaries between domain logic, infrastructure adapters, API handlers, persistence adapters, and event adapters. This is required so Codex, Claude Code, and human developers can implement and test the module incrementally without hidden coupling.

User-output note: any user-visible output related to AQELYN Knowledge Graph shall be written in clear language first and detailed technical language second. The system must always answer: what was found, why it matters, how AQELYN knows, and what the user can do next.

Government and enterprise note: AQELYN Knowledge Graph shall support auditability, exportable records, retention policy alignment, and review-ready evidence packages. The design supports private users, companies, managed service providers, and government environments without creating separate architectures.

028 - Reliability and Availability

Reliability and Availability for AQELYN Knowledge Graph shall be implemented in accordance with AQELYN Platform Architecture Baseline v1.0. The implementation must preserve traceability, deterministic behavior, explainability, and evidence-backed outputs.

This section defines engineering expectations for knowledge graph and provides the implementation team with stable guidance that should not be bypassed by local optimization or AI-generated redesign.

Implementation note: AQELYN Knowledge Graph must treat every decision as auditable. If the engine produces a finding, state transition, recommendation, event, or policy result, that output must reference the reason, source, timestamp, and confidence context. This supports AQELYN's user promise that security outcomes are understandable to non-experts and actionable by experts.

Quality note: AQELYN Knowledge Graph shall include code-level tests, schema-level tests, and documentation-level traceability. Any deviation discovered during implementation shall be raised as an Engineering Change Request rather than silently changing the architecture.

Operational note: AQELYN Knowledge Graph shall expose health state, readiness state, dependency state, and degraded-mode state. Operators must be able to understand whether the engine is fully operational, partially degraded, blocked by an upstream dependency, or intentionally disabled by policy.

Developer note: implementation of AQELYN Knowledge Graph shall follow clean boundaries between domain logic, infrastructure adapters, API handlers, persistence adapters, and event adapters. This is required so Codex, Claude Code, and human developers can implement and test the module incrementally without hidden coupling.

User-output note: any user-visible output related to AQELYN Knowledge Graph shall be written in clear language first and detailed technical language second. The system must always answer: what was found, why it matters, how AQELYN knows, and what the user can do next.

Government and enterprise note: AQELYN Knowledge Graph shall support auditability, exportable records, retention policy alignment, and review-ready evidence packages. The design supports private users, companies, managed service providers, and government environments without creating separate architectures.

029 - Threat Model

Threat Model for AQELYN Knowledge Graph shall be implemented in accordance with AQELYN Platform Architecture Baseline v1.0. The implementation must preserve traceability, deterministic behavior, explainability, and evidence-backed outputs.

This section defines engineering expectations for knowledge graph and provides the implementation team with stable guidance that should not be bypassed by local optimization or AI-generated redesign.

Implementation note: AQELYN Knowledge Graph must treat every decision as auditable. If the engine produces a finding, state transition, recommendation, event, or policy result, that output must reference the reason, source, timestamp, and confidence context. This supports AQELYN's user promise that security outcomes are understandable to non-experts and actionable by experts.

Quality note: AQELYN Knowledge Graph shall include code-level tests, schema-level tests, and documentation-level traceability. Any deviation discovered during implementation shall be raised as an Engineering Change Request rather than silently changing the architecture.

Operational note: AQELYN Knowledge Graph shall expose health state, readiness state, dependency state, and degraded-mode state. Operators must be able to understand whether the engine is fully operational, partially degraded, blocked by an upstream dependency, or intentionally disabled by policy.

Developer note: implementation of AQELYN Knowledge Graph shall follow clean boundaries between domain logic, infrastructure adapters, API handlers, persistence adapters, and event adapters. This is required so Codex, Claude Code, and human developers can implement and test the module incrementally without hidden coupling.

User-output note: any user-visible output related to AQELYN Knowledge Graph shall be written in clear language first and detailed technical language second. The system must always answer: what was found, why it matters, how AQELYN knows, and what the user can do next.

Government and enterprise note: AQELYN Knowledge Graph shall support auditability, exportable records, retention policy alignment, and review-ready evidence packages. The design supports private users, companies, managed service providers, and government environments without creating separate architectures.

030 - Secure Implementation Guidance

Secure Implementation Guidance for AQELYN Knowledge Graph shall be implemented in accordance with AQELYN Platform Architecture Baseline v1.0. The implementation must preserve traceability, deterministic behavior, explainability, and evidence-backed outputs.

This section defines engineering expectations for knowledge graph and provides the implementation team with stable guidance that should not be bypassed by local optimization or AI-generated redesign.

Implementation note: AQELYN Knowledge Graph must treat every decision as auditable. If the engine produces a finding, state transition, recommendation, event, or policy result, that output must reference the reason, source, timestamp, and confidence context. This supports AQELYN's user promise that security outcomes are understandable to non-experts and actionable by experts.

Quality note: AQELYN Knowledge Graph shall include code-level tests, schema-level tests, and documentation-level traceability. Any deviation discovered during implementation shall be raised as an Engineering Change Request rather than silently changing the architecture.

Operational note: AQELYN Knowledge Graph shall expose health state, readiness state, dependency state, and degraded-mode state. Operators must be able to understand whether the engine is fully operational, partially degraded, blocked by an upstream dependency, or intentionally disabled by policy.

Developer note: implementation of AQELYN Knowledge Graph shall follow clean boundaries between domain logic, infrastructure adapters, API handlers, persistence adapters, and event adapters. This is required so Codex, Claude Code, and human developers can implement and test the module incrementally without hidden coupling.

User-output note: any user-visible output related to AQELYN Knowledge Graph shall be written in clear language first and detailed technical language second. The system must always answer: what was found, why it matters, how AQELYN knows, and what the user can do next.

Government and enterprise note: AQELYN Knowledge Graph shall support auditability, exportable records, retention policy alignment, and review-ready evidence packages. The design supports private users, companies, managed service providers, and government environments without creating separate architectures.

031 - Testing Strategy

Testing shall include unit tests, integration tests, contract tests, security tests, performance tests, regression tests, and acceptance tests. Tests must be runnable locally and in CI/CD.

- Unit tests validate pure logic and boundary conditions.
- Integration tests validate engine interactions.
- Contract tests validate API and event schemas.
- Security tests validate authorization, input handling, and secret safety.

Implementation note: AQELYN Knowledge Graph must treat every decision as auditable. If the engine produces a finding, state transition, recommendation, event, or policy result, that output must reference the reason, source, timestamp, and confidence context. This supports AQELYN's user promise that security outcomes are understandable to non-experts and actionable by experts.

Quality note: AQELYN Knowledge Graph shall include code-level tests, schema-level tests, and documentation-level traceability. Any deviation discovered during implementation shall be raised as an Engineering Change Request rather than silently changing the architecture.

Operational note: AQELYN Knowledge Graph shall expose health state, readiness state, dependency state, and degraded-mode state. Operators must be able to understand whether the engine is fully operational, partially degraded, blocked by an upstream dependency, or intentionally disabled by policy.

Developer note: implementation of AQELYN Knowledge Graph shall follow clean boundaries between domain logic, infrastructure adapters, API handlers, persistence adapters, and event adapters. This is required so Codex, Claude Code, and human developers can implement and test the module incrementally without hidden coupling.

User-output note: any user-visible output related to AQELYN Knowledge Graph shall be written in clear language first and detailed technical language second. The system must always answer: what was found, why it matters, how AQELYN knows, and what the user can do next.

Government and enterprise note: AQELYN Knowledge Graph shall support auditability, exportable records, retention policy alignment, and review-ready evidence packages. The design supports private users, companies, managed service providers, and government environments without creating separate architectures.

032 - Validation Strategy

Validation Strategy for AQELYN Knowledge Graph shall be implemented in accordance with AQELYN Platform Architecture Baseline v1.0. The implementation must preserve traceability, deterministic behavior, explainability, and evidence-backed outputs.

This section defines engineering expectations for knowledge graph and provides the implementation team with stable guidance that should not be bypassed by local optimization or AI-generated redesign.

Implementation note: AQELYN Knowledge Graph must treat every decision as auditable. If the engine produces a finding, state transition, recommendation, event, or policy result, that output must reference the reason, source, timestamp, and confidence context. This supports AQELYN's user promise that security outcomes are understandable to non-experts and actionable by experts.

Quality note: AQELYN Knowledge Graph shall include code-level tests, schema-level tests, and documentation-level traceability. Any deviation discovered during implementation shall be raised as an Engineering Change Request rather than silently changing the architecture.

Operational note: AQELYN Knowledge Graph shall expose health state, readiness state, dependency state, and degraded-mode state. Operators must be able to understand whether the engine is fully operational, partially degraded, blocked by an upstream dependency, or intentionally disabled by policy.

Developer note: implementation of AQELYN Knowledge Graph shall follow clean boundaries between domain logic, infrastructure adapters, API handlers, persistence adapters, and event adapters. This is required so Codex, Claude Code, and human developers can implement and test the module incrementally without hidden coupling.

User-output note: any user-visible output related to AQELYN Knowledge Graph shall be written in clear language first and detailed technical language second. The system must always answer: what was found, why it matters, how AQELYN knows, and what the user can do next.

Government and enterprise note: AQELYN Knowledge Graph shall support auditability, exportable records, retention policy alignment, and review-ready evidence packages. The design supports private users, companies, managed service providers, and government environments without creating separate architectures.

033 - Acceptance Criteria

Acceptance Criteria for AQELYN Knowledge Graph shall be implemented in accordance with AQELYN Platform Architecture Baseline v1.0. The implementation must preserve traceability, deterministic behavior, explainability, and evidence-backed outputs.

This section defines engineering expectations for knowledge graph and provides the implementation team with stable guidance that should not be bypassed by local optimization or AI-generated redesign.

Implementation note: AQELYN Knowledge Graph must treat every decision as auditable. If the engine produces a finding, state transition, recommendation, event, or policy result, that output must reference the reason, source, timestamp, and confidence context. This supports AQELYN's user promise that security outcomes are understandable to non-experts and actionable by experts.

Quality note: AQELYN Knowledge Graph shall include code-level tests, schema-level tests, and documentation-level traceability. Any deviation discovered during implementation shall be raised as an Engineering Change Request rather than silently changing the architecture.

Operational note: AQELYN Knowledge Graph shall expose health state, readiness state, dependency state, and degraded-mode state. Operators must be able to understand whether the engine is fully operational, partially degraded, blocked by an upstream dependency, or intentionally disabled by policy.

Developer note: implementation of AQELYN Knowledge Graph shall follow clean boundaries between domain logic, infrastructure adapters, API handlers, persistence adapters, and event adapters. This is required so Codex, Claude Code, and human developers can implement and test the module incrementally without hidden coupling.

User-output note: any user-visible output related to AQELYN Knowledge Graph shall be written in clear language first and detailed technical language second. The system must always answer: what was found, why it matters, how AQELYN knows, and what the user can do next.

Government and enterprise note: AQELYN Knowledge Graph shall support auditability, exportable records, retention policy alignment, and review-ready evidence packages. The design supports private users, companies, managed service providers, and government environments without creating separate architectures.

034 - Requirements Matrix

ID	Requirement	Priority	Verification
FR-001	Capability 1 for Knowledge Graph	Mandatory	Test + Review
FR-002	Capability 2 for Knowledge Graph	Mandatory	Test + Review
FR-003	Capability 3 for Knowledge Graph	Mandatory	Test + Review
FR-004	Capability 4 for Knowledge Graph	Mandatory	Test + Review
FR-005	Capability 5 for Knowledge Graph	Mandatory	Test + Review
FR-006	Capability 6 for Knowledge Graph	Mandatory	Test + Review
FR-007	Capability 7 for Knowledge Graph	Mandatory	Test + Review
FR-008	Capability 8 for Knowledge Graph	Mandatory	Test + Review
FR-009	Capability 9 for Knowledge Graph	Mandatory	Test + Review
FR-010	Capability 10 for Knowledge Graph	Mandatory	Test + Review

035 - Traceability Matrix

Requirement	Component	Test	Evidence
FR-001	Knowledge Graph Service Layer	automated test	evidence reference + CI artifact
FR-002	Knowledge Graph Service Layer	automated test	evidence reference + CI artifact
FR-003	Knowledge Graph Service Layer	automated test	evidence reference + CI artifact
FR-004	Knowledge Graph Service Layer	automated test	evidence reference + CI artifact
FR-005	Knowledge Graph Service Layer	automated test	evidence reference + CI artifact
FR-006	Knowledge Graph Service Layer	automated test	evidence reference + CI artifact
FR-007	Knowledge Graph Service Layer	automated test	evidence reference + CI artifact
FR-008	Knowledge Graph Service Layer	automated test	evidence reference + CI artifact
FR-009	Knowledge Graph Service Layer	automated test	evidence reference + CI artifact
FR-010	Knowledge Graph Service Layer	automated test	evidence reference + CI artifact

036 - Example Artifacts

Example artifacts included with this archive demonstrate canonical payloads, event envelopes, review checklists, and implementation-ready acceptance examples. They are illustrative and may be expanded during coding without changing the architecture.

Implementation note: AQELYN Knowledge Graph must treat every decision as auditable. If the engine produces a finding, state transition, recommendation, event, or policy result, that output must reference the reason, source, timestamp, and confidence context. This supports AQELYN's user promise that security outcomes are understandable to non-experts and actionable by experts.

Quality note: AQELYN Knowledge Graph shall include code-level tests, schema-level tests, and documentation-level traceability. Any deviation discovered during implementation shall be raised as an Engineering Change Request rather than silently changing the architecture.

Operational note: AQELYN Knowledge Graph shall expose health state, readiness state, dependency state, and degraded-mode state. Operators must be able to understand whether the engine is fully operational, partially degraded, blocked by an upstream dependency, or intentionally disabled by policy.

Developer note: implementation of AQELYN Knowledge Graph shall follow clean boundaries between domain logic, infrastructure adapters, API handlers, persistence adapters, and event adapters. This is required so Codex, Claude Code, and human developers can implement and test the module incrementally without hidden coupling.

User-output note: any user-visible output related to AQELYN Knowledge Graph shall be written in clear language first and detailed technical language second. The system must always answer: what was found, why it matters, how AQELYN knows, and what the user can do next.

Government and enterprise note: AQELYN Knowledge Graph shall support auditability, exportable records, retention policy alignment, and review-ready evidence packages. The design supports private users, companies, managed service providers, and government environments without creating separate architectures.

037 - Engineering Journal

This archive was regenerated to align early AQELYN foundation packages with the later FULL_COMPLETE publication standard. The regeneration removes legacy AQELYN codename remnants, expands the implementation guidance, and brings PDF/HTML artifacts into consistent layout with EA-0010 through EA-0063.

Implementation note: AQELYN Knowledge Graph must treat every decision as auditable. If the engine produces a finding, state transition, recommendation, event, or policy result, that output must reference the reason, source, timestamp, and confidence context. This supports AQELYN's user promise that security outcomes are understandable to non-experts and actionable by experts.

Quality note: AQELYN Knowledge Graph shall include code-level tests, schema-level tests, and documentation-level traceability. Any deviation discovered during implementation shall be raised as an Engineering Change Request rather than silently changing the architecture.

Operational note: AQELYN Knowledge Graph shall expose health state, readiness state, dependency state, and degraded-mode state. Operators must be able to understand whether the engine is fully operational, partially degraded, blocked by an upstream dependency, or intentionally disabled by policy.

Developer note: implementation of AQELYN Knowledge Graph shall follow clean boundaries between domain logic, infrastructure adapters, API handlers, persistence adapters, and event adapters. This is required so Codex, Claude Code, and human developers can implement and test the module incrementally without hidden coupling.

User-output note: any user-visible output related to AQELYN Knowledge Graph shall be written in clear language first and detailed technical language second. The system must always answer: what was found, why it matters, how AQELYN knows, and what the user can do next.

Government and enterprise note: AQELYN Knowledge Graph shall support auditability, exportable records, retention policy alignment, and review-ready evidence packages. The design supports private users, companies, managed service providers, and government environments without creating separate architectures.

038 - Deployment Considerations

Deployment Considerations for AQELYN Knowledge Graph shall be implemented in accordance with AQELYN Platform Architecture Baseline v1.0. The implementation must preserve traceability, deterministic behavior, explainability, and evidence-backed outputs.

This section defines engineering expectations for knowledge graph and provides the implementation team with stable guidance that should not be bypassed by local optimization or AI-generated redesign.

Implementation note: AQELYN Knowledge Graph must treat every decision as auditable. If the engine produces a finding, state transition, recommendation, event, or policy result, that output must reference the reason, source, timestamp, and confidence context. This supports AQELYN's user promise that security outcomes are understandable to non-experts and actionable by experts.

Quality note: AQELYN Knowledge Graph shall include code-level tests, schema-level tests, and documentation-level traceability. Any deviation discovered during implementation shall be raised as an Engineering Change Request rather than silently changing the architecture.

Operational note: AQELYN Knowledge Graph shall expose health state, readiness state, dependency state, and degraded-mode state. Operators must be able to understand whether the engine is fully operational, partially degraded, blocked by an upstream dependency, or intentionally disabled by policy.

Developer note: implementation of AQELYN Knowledge Graph shall follow clean boundaries between domain logic, infrastructure adapters, API handlers, persistence adapters, and event adapters. This is required so Codex, Claude Code, and human developers can implement and test the module incrementally without hidden coupling.

User-output note: any user-visible output related to AQELYN Knowledge Graph shall be written in clear language first and detailed technical language second. The system must always answer: what was found, why it matters, how AQELYN knows, and what the user can do next.

Government and enterprise note: AQELYN Knowledge Graph shall support auditability, exportable records, retention policy alignment, and review-ready evidence packages. The design supports private users, companies, managed service providers, and government environments without creating separate architectures.

039 - Codex and AI Implementation Guidance

Codex and AI Implementation Guidance for AQELYN Knowledge Graph shall be implemented in accordance with AQELYN Platform Architecture Baseline v1.0. The implementation must preserve traceability, deterministic behavior, explainability, and evidence-backed outputs.

This section defines engineering expectations for knowledge graph and provides the implementation team with stable guidance that should not be bypassed by local optimization or AI-generated redesign.

Implementation note: AQELYN Knowledge Graph must treat every decision as auditable. If the engine produces a finding, state transition, recommendation, event, or policy result, that output must reference the reason, source, timestamp, and confidence context. This supports AQELYN's user promise that security outcomes are understandable to non-experts and actionable by experts.

Quality note: AQELYN Knowledge Graph shall include code-level tests, schema-level tests, and documentation-level traceability. Any deviation discovered during implementation shall be raised as an Engineering Change Request rather than silently changing the architecture.

Operational note: AQELYN Knowledge Graph shall expose health state, readiness state, dependency state, and degraded-mode state. Operators must be able to understand whether the engine is fully operational, partially degraded,

blocked by an upstream dependency, or intentionally disabled by policy.

Developer note: implementation of AQELYN Knowledge Graph shall follow clean boundaries between domain logic, infrastructure adapters, API handlers, persistence adapters, and event adapters. This is required so Codex, Claude Code, and human developers can implement and test the module incrementally without hidden coupling.

User-output note: any user-visible output related to AQELYN Knowledge Graph shall be written in clear language first and detailed technical language second. The system must always answer: what was found, why it matters, how AQELYN knows, and what the user can do next.

Government and enterprise note: AQELYN Knowledge Graph shall support auditability, exportable records, retention policy alignment, and review-ready evidence packages. The design supports private users, companies, managed service providers, and government environments without creating separate architectures.

040 - Publication Readiness

EA-0005 is publication-ready when the master Markdown, PDF, HTML, README, manifest, requirements matrix, traceability matrix, engineering journal, examples, diagrams, and release ZIP are present and hash-verified.

Implementation note: AQELYN Knowledge Graph must treat every decision as auditable. If the engine produces a finding, state transition, recommendation, event, or policy result, that output must reference the reason, source, timestamp, and confidence context. This supports AQELYN's user promise that security outcomes are understandable to non-experts and actionable by experts.

Quality note: AQELYN Knowledge Graph shall include code-level tests, schema-level tests, and documentation-level traceability. Any deviation discovered during implementation shall be raised as an Engineering Change Request rather than silently changing the architecture.

Operational note: AQELYN Knowledge Graph shall expose health state, readiness state, dependency state, and degraded-mode state. Operators must be able to understand whether the engine is fully operational, partially degraded, blocked by an upstream dependency, or intentionally disabled by policy.

Developer note: implementation of AQELYN Knowledge Graph shall follow clean boundaries between domain logic, infrastructure adapters, API handlers, persistence adapters, and event adapters. This is required so Codex, Claude Code, and human developers can implement and test the module incrementally without hidden coupling.

User-output note: any user-visible output related to AQELYN Knowledge Graph shall be written in clear language first and detailed technical language second. The system must always answer: what was found, why it matters, how AQELYN knows, and what the user can do next.

Government and enterprise note: AQELYN Knowledge Graph shall support auditability, exportable records, retention policy alignment, and review-ready evidence packages. The design supports private users, companies, managed service providers, and government environments without creating separate architectures.